Univerzita Karlova

Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země

Jan Bartůšek, Petr Havel

Aplikace počítačové kartografie

Digitální model terénu a jeho analýzy

1. Zadání

Vstup: množina $P = \{p_1, ..., p_n\}$, $p_i = \{x_i, y_i, z_i\}$.

Výstup: polyedrický DMT nad množinou P představovaný vrstevnicemi doplněný vizualizací sklonu trojúhelníků a jejich expozicí.

Metodou inkrementální konstrukce vytvořte nad množinou P vstupních bodů 2D Delaunay triangulaci. Jako vstupní data použijte existující geodetická data (alespoň 300 bodů) popř. navrhněte algoritmus pro generování syntetických vstupních dat představujících významné terénní tvary (kupa, údolí, spočinek, hřbet, ...).

Vstupní množiny bodů včetně níže uvedených výstupů vhodně vizualizujte. Grafické rozhraní realizujte s využitím frameworku QT. Dynamické datové struktury implementujte s využitím STL.

Nad takto vzniklou triangulací vygenerujte polyedrický digitální model terénu. Dále proveďte tyto analýzy:

- S využitím lineární interpolace vygenerujte vrstevnice se *zadaným krokem* a v *zadaném intervalu*, proveďte jejich vizualizaci s rozlišením zvýrazněných vrstevnic.
- Analyzujte sklon digitálního modelu terénu, jednotlivé trojúhelníky vizualizujte v závislosti na jejich sklonu.
- Analyzujte expozici digitálního modelu terénu, jednotlivé trojúhelníky vizualizujte v závislosti na jejich expozici ke světové straně.

Zhodnoťte výsledný digitální model terénu z kartografického hlediska, zamyslete se nad slabinami algoritmu založeného na 2D Delaunay triangulaci. Ve kterých situacích (různé terénní tvary) nebude dávat vhodné výsledky? Tyto situace graficky znázorněte.

Zhodnocení činnosti algoritmu včetně ukázek proveďte alespoň na **3 strany** formátu A4.

2. Údaje o bonusových úlohách + počet bodů

Aplikace nad rámec základního zadání (20 b) obsahuje implementaci následujících bonusových úloh v celkové hodnotě 45 bodů:

Triangulace nekonvexní oblasti zadané polygonem (+5 b): Ořezání sítě pomocí Ray-Casting algoritmu aplikovaného na těžiště trojúhelníků.

Výběr barevných stupnic (+3 b): Aplikace barevného modelu HSV pro plynulé a logické přechody barev sklonu a plné spektrum expozice.

Automatický popis vrstevnic (+3 b): Výpočet polohy a vykreslení textových štítků s hodnotou nadmořské výšky.

Popis vrstevnic respektující kartografické zásady (+10 b): Zvýraznění intervalových vrstevnic, rotace textu orientovaná rovnoběžně se svahem (čitelná orientace) a ochranná zóna proti překryvu textů.

Generování terénních tvarů (+10 b): Matematické modely (Gaussova křivka, hyperbolický paraboloid) pro syntetické tvary: kupa, údolí, hřbet, spočinek.

3D vizualizace terénu (+10 b): Prostorové zobrazení pomocí izometrické projekce a Malířova algoritmu (Painter's algorithm).

Barevná hypsometrie (+5 b): Dynamické mapování nadmořské výšky (Z) do barevného spektra HSV.

Celkový možný počet získaných bodů: 65

3. Popis a rozbor problému a vzorce

Model je reprezentován jako 2.5D povrch, kde triangulace probíhá v rovině XY. Pro výpočet parametrů využíváme normálový vektor $\vec{n}$.

Vektorový součin (Normála):

$$\vec{n} = \vec{u} \times \vec{v} = (u_y v_z - u_z v_y, u_z v_x - u_x v_z, u_x v_y - u_y v_x) = (n_x, n_y, n_z)$$

Sklon svahu (Slope):

$$\text{Slope} = \arctan\left(\frac{\sqrt{n_x^2 + n_y^2}}{|n_z|}\right) \cdot \frac{180}{\pi}$$

Expozice svahu (Aspect):

$$\text{Aspect} = \text{atan2}(n_x, n_y) \cdot \frac{180}{\pi}$$

Lineární interpolace vrstevnic:

$$x = x_A + \frac{Z_0 - z_A}{z_B - z_A} \cdot (x_B - x_A)$$

Popisy algoritmů formálním jazykem

Algoritmus: Graham Scan pro konvexní obálku 1: Nalezení pivota $q = \min(y_i)$, $q \in H$ 2: Setřídění $\forall p_i \in S$ dle polárního úhlu $\omega_i = \angle(p_i, q, x)$ 3: Inicializace zásobníku: $S \leftarrow q, S \leftarrow p_1$ 4: Pro všechny zbývající setříděné body p_j: 5: Zatímco p_j netvoří levotočivou zatáčku s posledními dvěma body v S: 6: S.pop() (odstranění reflexního vrcholu) 7: S.push(p_j) 8: Vrať prvky v zásobníku S jako konvexní obálku H.

Algoritmus: Minimum Area Enclosing Rectangle (MAER) 1: Najdi H = CH(P) 2: Inicializuj minimální plochu A = ∞ a obdélník R 3: Opakuj pro každou hranu $e \in H$: 4: Spočti směrnici \sigma hrany e 5: Otoč body P o úhel -\sigma: P_0 = R($-\sigma$)P 6: Najdi Bounding Box pro P_0 *a urči jeho plochu A* 7: Pokud A < A 8: A = A, $\underline{\sigma} = \sigma$, ulož aktuální Bounding Box 9: Zrotuj uložený Bounding Box zpět o úhel $\underline{\sigma}$ do geografické polohy.

4. Problematické situace a jejich rozbor (Simplexy)

4.1 Delaunay triangulace (AEL)

Algoritmus inkrementální konstrukce využívá seznam aktivních hran (AEL). Pro každou hranu hledá bod v levé polorovině, který splňuje Delaunayovu podmínku (v kružnici opsané trojúhelníku neleží žádný jiný bod). Tím je zaručena optimalizace vnitřních úhlů trojúhelníků.

4.2 Ořez polygonem (Ray-Casting)

Algoritmus určuje příslušnost bodu k polygonu. Z testovaného bodu (těžiště trojúhelníka) vedeme paprsek. Pokud je počet průsečíků paprsku s hranami polygonu lichý, bod (a tedy celý trojúhelník) leží uvnitř a je v modelu ponechán.

5. Problematické situace a jejich rozbor (Simplexy)

A. Vertikální stěny (Dělení nulou)

Při $n_z = 0$ by došlo k pádu výpočtu sklonu. V triangle.py je ošetřeno:

```
# 3. Compute slope in degrees
if nz != 0:
    self.__slope = atan(norm_xy / abs(nz)) * (180 / pi)
else:
    self.__slope = 90.0 # Vertical wall
```

B. Vodorovné plochy (Singularita expozice)

Rovina nemá směr sklonu. V triangle.py vracíme hodnotu -1:

```
# 4. Compute aspect (azimuth) in degrees
if norm_xy > 1e-9:
    aspect_rad = atan2(nx, ny)
    self.__aspect = aspect_rad * (180 / pi)
    # Convert to positive angles (0 - 360)
    if self.__aspect < 0:
        self.__aspect += 360
else:
    self.__aspect = -1.0
```

C. Nekonvexní oblasti (Ořez sítě)

Odstranění nežádoucích hran na okraji sítě v algorithms.py pomocí testu těžišť:

```
for t in dt_triangles:
    # Calculate triangle centroid
    cx = (t.getP1().x() + t.getP2().x() + t.getP3().x()) / 3.0
    cy = (t.getP1().y() + t.getP2().y() + t.getP3().y()) / 3.0
    from PyQt6.QtCore import QPointF
    centroid = QPointF(cx, cy)

    # Keep triangle if its centroid is inside the polygon
    if self.isPointInPolygon(centroid, polygon):
        clipped_triangles.append(t)
```

D. Kontrast analýzy sklonu

U mírných terénů by pevná barevná škála byla nečitelná. V draw.py používáme dynamickou normalizaci:

```
s_min = min(t.getSlope() for t in self.__triangles)
s_max = max(t.getSlope() for t in self.__triangles)
ds = max(s_max - s_min, 0.1)

for t in self.__triangles:
    slope = t.getSlope()
    norm_s = (slope - s_min) / ds
    hue = int(120 - norm_s * 120)
    color = QColor.fromHsv(max(0, min(120, hue)), 255, 255)
```

6. Vstupní data

Vstupní data aplikace představují diskrétní množinu bodů v trojrozměrném prostoru, které jsou v paměti reprezentovány specifickými datovými strukturami.

Datový formát: Základním stavebním prvkem je třída QPoint3DF, která dědí z QPointF (souřadnice x, y typu float). Tato třída rozšiřuje standardní bod o privátní atribut __z, reprezentující nadmořskou výšku.

Souřadnicový systém: Aplikace pracuje v obrazovkovém souřadnicovém systému Qt, kde počátek [0,0] leží v levém horním rohu plátna. Osa x roste doprava, osa y roste směrem dolů. Výška z je pro vizualizaci uvažována jako nezávislá hodnota.

Zdroje dat:

Interaktivní zadávání: Uživatel vkládá body levým tlačítkem myši. Souřadnice x, y jsou dány polohou kurzoru, výška z je generována pseudonáhodně v intervalu $\langle 200, 600 \rangle$ pomocí modulu random.

Syntetické generování: Metoda generateTerrain v souboru algorithms.py vytváří pravidelnou síť bodů (grid) s krokem 20 px. Výška každého bodu je pak vypočtena na základě zvoleného morfologického typu:

Kupa (Hill): Využívá Gaussovu funkci:

$$z = h \cdot \exp\left(-\left(dist^2/(2 \cdot \sigma^2)\right)\right)$$

Sedlo (Saddle): Využívá rovnici hyperbolického paraboloidu.

Ořezová data: Samostatnou entitou je seznam __polygon ve třídě Draw, který uchovává vrcholy nekonvexní hranice zadané pravým tlačítkem myši.

7. Výstupní data (Vizualizace a formáty)

Výstupem aplikace je grafický obraz generovaný v reálném čase pomocí třídy QPainter.

Vektorové vrstvy:

Delaunayova síť: Vykreslována jako úsečky (objekty Edge) spojující vrcholy trojúhelníků. Používá se zelená linka o tloušťce 1 px.

Vrstevnice: Výsledkem interpolace je seznam hran __contours. Jsou rozlišeny základní vrstevnice (šedé, tloušťka 1 px) a zdůrazněné vrstevnice (černé, tloušťka 2 px, interval 100 m).

Plošné (analytické) vrstvy:

Jsou vykreslovány jako vyplněné polygony trojúhelníků pomocí metody drawPolygon.

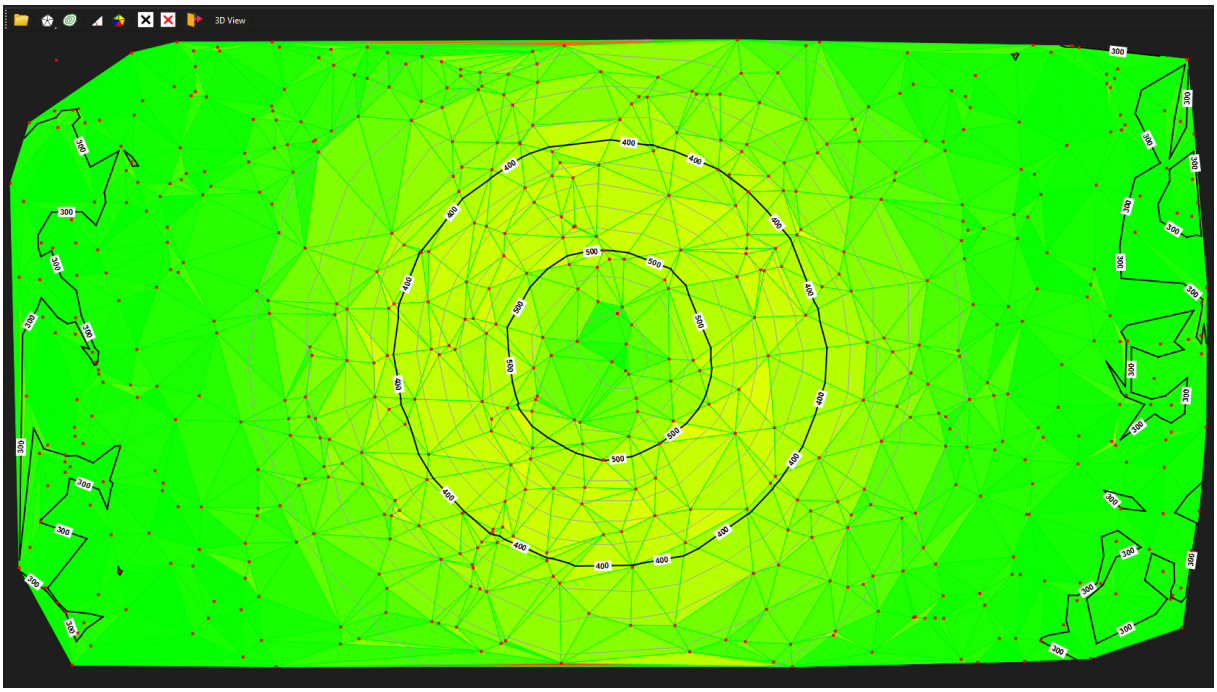
Barevné modely: Všechny analýzy (Sklon, Expozice, Hypsometrie) využívají model HSV (Hue, Saturation, Value). Hue (odstín) je mapován na vypočtenou hodnotu analýzy, zatímco Saturation a Value jsou drženy na maximu (255) pro dosažení vysokého kontrastu.

3D reprezentace:

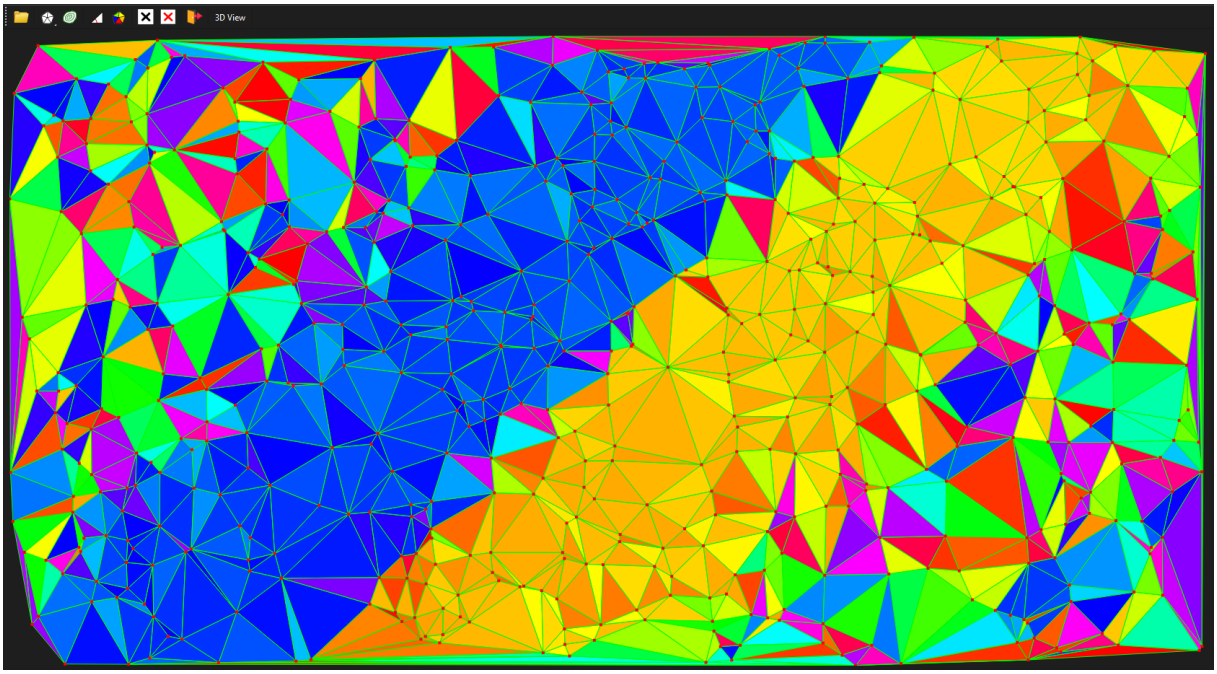
Výstupem je izometrický pohled transformovaný z 3D souřadnic do 2D plátna.

Aby byla zajištěna korektní viditelnost ploch, je implementován Malířův algoritmus, který trojúhelníky před vykreslením seřadí podle jejich vzdálenosti od pozorovatele (v kódu pomocí součtu souřadnic $x + y$).

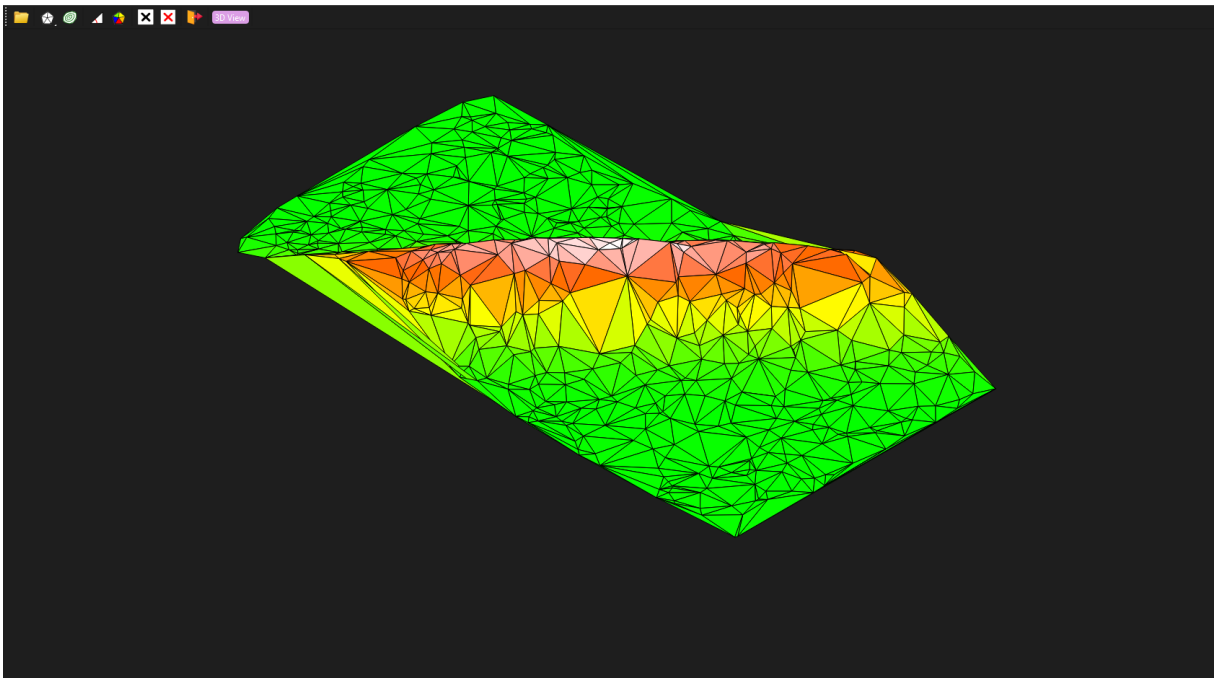
8. Printscreen vytvořené aplikace



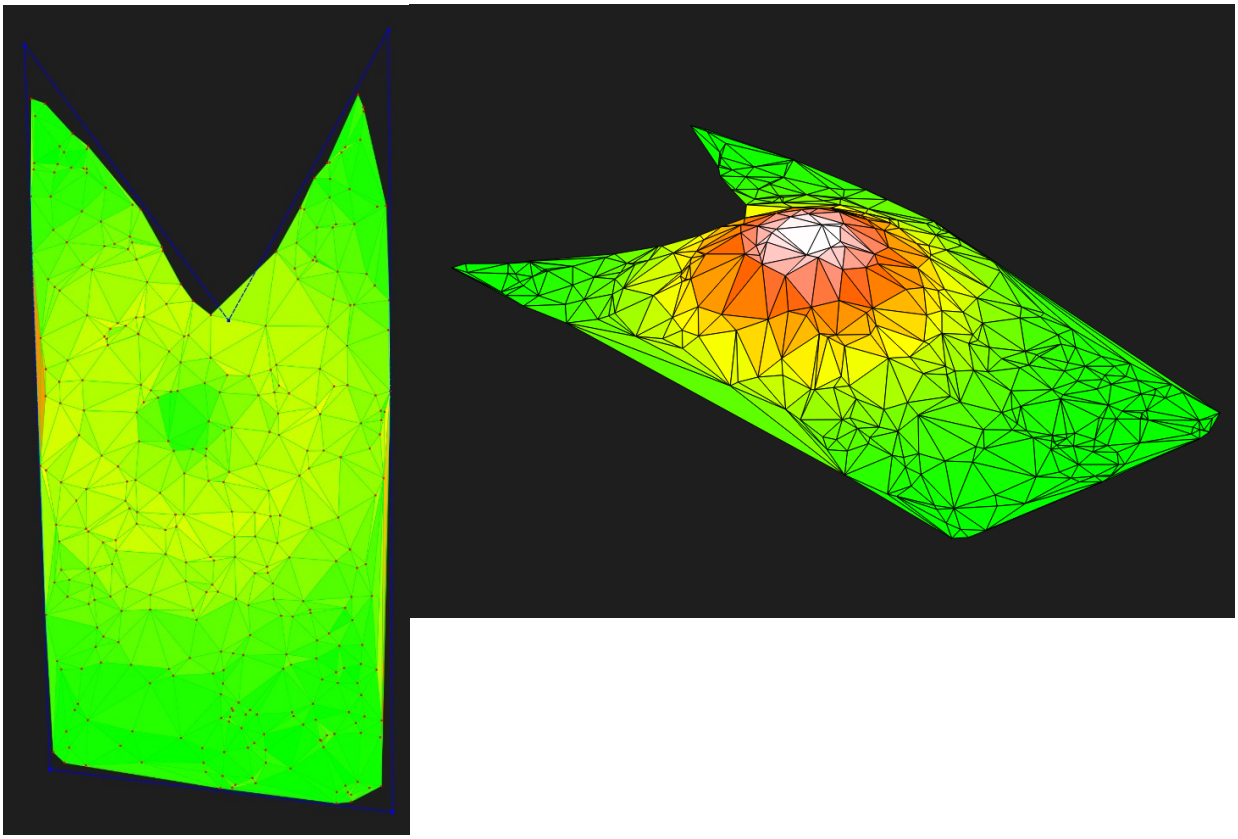
Obr. 1: Hlavní okno, hypsometrie + popsané vrstevnice.



Obr. 2: Analýza expozice (Aspect) – hřbet s barevným azimutem



Obr. 3: 3D vizualizace



Obr. 4: Modrý nekonvexní polygon a oříznutá síť

9. Dokumentace (Podrobný popis tříd a metod)

9.1 Třída QPoint3DF (soubor qpoint3df.py)

Základní datový typ pro uchování 3D souřadnic.

__init__(self, x, y, z): Konstruktor inicializující 2D polohu a výšku.

z(self): Getter pro získání výškové souřadnice.

9.2 Třída Edge (soubor edge.py)

Reprezentuje orientovanou hranu triangulace.

__start, __end: Počáteční a koncový bod typu QPoint3DF.

switchOrientation(self): Vrací novou hranu s prohozenými body (důležité pro algoritmus AEL).

__eq__(self, other): Operátor rovnosti pro identifikaci duplicitních hran v síti.

9.3 Třída Triangle (soubor triangle.py)

Klíčová třída pro prostorové analýzy.

calculateGeometry(self): Metoda volaná v konstruktoru. Počítá směrové vektory hran, jejich vektorový součin (normálu) a následně **Sklon** a **Expozici**. Obsahuje ošetření pro svislé stěny ($n_z = 0$) a vodorovné roviny.

getSlope(), getAspect(): Gettery pro výsledky analýz.

9.4 Třída Algorithms (soubor algorithms.py)

Knihovní třída pro geometrické výpočty.

getPointLinePosition(self, a, b, p): Určuje polohu bodu vůči orientované přímce (využívá determinant).

createDT(self, points): Hlavní metoda pro inkrementální konstrukci Delaunayovy triangulace.

createContourLines(self, triangles, dz): Generuje vrstevnice lineární interpolací na hranách trojúhelníků.

isPointInPolygon(self, p, polygon): Implementace Ray-Casting algoritmu pro test bodu v polygonu.

clipDT(self, edges, triangles, polygon): Ořezává vytvořenou síť podle nekonvexní hranice.

9.5 Třída Draw (soubor draw.py)

Zajišťuje interakci a vykreslování.

mousePressEvent(self, e): Rozlišuje levý klik (přidání bodu terénu) a pravý klik (vrchol ořezového polygonu).

paintEvent(self, e): Centrální metoda vykreslování. Obsahuje logiku pro přepínání vrstev (view_dt, view_slope, atd.).

drawSlope(self, qp), drawAspect(self, qp): Metody pro plošné vybarvení terénu podle sklonu a orientace.

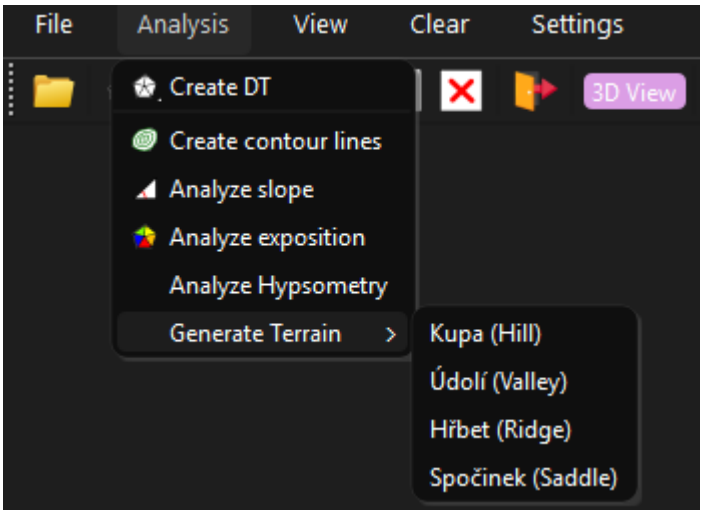
projectTo3D(self, p, w, h): Matematická transformace bodu do izometrického zobrazení.

9.6 Třída Ui_MainWindow (soubor MainForm.py)

Definuje uživatelské rozhraní a logiku ovládání.

generateTerrainClick(self, type): Obsluhuje generování syntetických tvarů a automaticky filtruje body, které leží mimo nakreslený polygon.

createDTClick(self): Propojuje UI s algoritmem triangulace a následným ořezem sítě.



Obr. 5: Detailní pohled na panel nástrojů a menu aplikace pro ilustraci UI popsaného v kap. 9.6

10. Závěr, možné či neřešené problémy, náměty na vylepšení

10.1 Závěr

Cílem projektu bylo vytvořit komplexní nástroj pro generování a analýzu digitálního modelu terénu. Tento cíl byl plně dosažen. Aplikace úspěšně implementuje inkrementální algoritmus Delaunayovy triangulace (AEL), na který navazují algoritmy pro generování vrstevnic a plošné prostorové analýzy.

Nad rámec základního zadání se podařilo úspěšně integrovat 7 bonusových úloh (v celkové hodnotě 45 bodů), které výrazně zvyšují kartografickou a analytickou hodnotu programu. Díky dynamické barevné škále HSV, pokročilému algoritmu pro inteligentní popisování vrstevnic (s ohledem na čitelnost a překryv) a implementaci ořezu pomocí Ray-Casting algoritmu se z jednoduchého geometrického skriptu stal robustní vizualizační nástroj.

10.2 Neřešené problémy a teoretické limity

Během vývoje a testování nad syntetickými i ručně zadávanými daty byly identifikovány následující limity aktuálního řešení:

1. **Absence usměrněné triangulace (CDT):**

Standardní Delaunayova triangulace optimalizuje tvar trojúhelníků (maximalizuje minimální úhel), ale zcela ignoruje geomorfologii terénu. V přírodě se vyskytují tzv. povinné hrany (breaklines), jako jsou koryta potoků, hřbety hor nebo ostré hrany silnic. Náš aktuální algoritmus tyto přirozené zlomové linie nerespektuje a běžně vytvoří trojúhelník, který údolí nebo hřbet "překlene". Tím dochází k nežádoucímu vyhlazení terénu a zkreslení analýz sklonu.

2. **Limitace 2.5D modelu:**

Zvolená datová struktura mapuje pro každou souřadnici $[X, Y]$ pouze jednu výškovou hodnotu Z. Z matematické podstaty tohoto 2.5D modelu tak nelze modelovat skutečně 3D útvary, jako jsou skalní převisy, jeskyně nebo kolmé tunely. Při pokusu o zadání takových bodů algoritmus selhává, nebo je spojí do nerealistické svislé stěny.

3. **Výpočetní složitost pro masivní datasety:**

Současná implementace vyhledávání bodů a určování polohy (např. metoda getNearestPoint) prochází iterativně celou množinu. Pro datové sady o velikosti stovek bodů je běh v jazyce Python okamžitý, avšak pro statisíce bodů (např. data z laserového skenování LiDAR) by kvadratická časová složitost $O(n^2)$ vedla k nepřiměřeně dlouhým výpočetním časům.

10.3 Náměty na vylepšení

Na základě výše zmíněných limitů se nabízejí následující směry pro budoucí rozvoj aplikace:

Implementace Constrained Delaunay Triangulation (CDT): Rozšíření uživatelského rozhraní o možnost kreslení zlomových linií (polygony/polylinie), do kterých by algoritmus nesměl vkládat křížící se hrany.

Prostorové indexování: Zavedení pokročilých datových struktur, jako je K-D strom (K-D Tree) nebo QuadTree, pro optimalizaci vyhledávání nejbližších sousedů a bodů v polorovině, čímž by se časová složitost snížila k $O(n \log n)$.

Pokročilé 3D zobrazení: Nahrazení izometrické projekce a softwarového Malířova algoritmu za plnohodnotné hardwarově akcelerované řešení (např. integrace knihovny OpenGL). To by umožnilo interaktivní rotaci terénu v prostoru pomocí myši a dynamické osvětlení (stínování) modelu.

Podpora standardizovaných GIS formátů: Přidání funkcí pro import a export reálných výškových bodů ve formátech ASCII XYZ, .las nebo ESRI Shapefile, což by aplikaci posunulo blíže k reálnému nasazení v praxi.

11. Seznam literatury

1. BAYER, Tomáš. *Rovinné triangulace a jejich využití*. Přednášky z předmětu Algoritmy v digitální kartografii. Praha: Katedra aplikované geoinformatiky a kartografie, Přírodovědecká fakulta Univerzity Karlovy.